# SmartLib – College Library Management System

A Course Project Report

*Submitted to the APJ Abdul Kalam Technological University in partial fulfillment of requirements for the award of degree*

**Bachelor of Technology**

in

*Computer Science and Engineering*

by


**Derin Jose Sanjith**
Register No. TVE25CS048

**Jenil Prince**
Register No. TVE25CS064

**Prathyush**
Register No. TVE25CS105

**Abhinand V P**
Register No. LTVE25CS142

**Adhithyan K S**
Register No. LTVE25CS143

Semester: S3

Under the guidance of

**Prof. Leena V.A**

Department of Computer Science and Engineering






**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**COLLEGE OF ENGINEERING TRIVANDRUM, THIRUVANANTHAPURAM**


October 2026

# Abstract

Academic libraries serve as the intellectual cornerstone of collegiate institutions, coordinating the acquisition, cataloging, circulation, and preservation of scholarly assets. Despite widespread digitization across universities, many institutional libraries continue to grapple with antiquated management software or error-prone manual ledgers. Common systemic drawbacks include cumbersome book circulation workflows, unmonitored overdue loans leading to substantial fee leakages, double-booking race conditions during manual reservations, and the complete absence of a secure, transparent self-service portal for enrolled students.

To resolve these challenges, this report presents **SmartLib**, a robust, desktop-based College Library Management System developed using Java SE 17, Swing/AWT graphical user interfaces, JDBC, and a relational MySQL database. Engineered in accordance with the curricular requirements of course **PBCST304 (Object Oriented Programming)**, SmartLib strictly adheres to multi-tier layered software architectural patterns, embodying rigorous Separation of Concerns (SoC) and the Dependency Inversion Principle (DIP). The architecture cleanly delineates responsibilities across four distinct layers: a responsive Presentation Layer employing Swing's `CardLayout` navigation model, a Business Service Layer encapsulating institutional policies, an abstracted Data Access Object (DAO) Layer centered around a generic `CrudDAO<T, ID>` base interface, and a persistent MySQL relational database.

SmartLib introduces functional operational features that streamline collegiate library administration. A unified authentication mechanism with 12-round BCrypt cryptographic password hashing provides role-based access control for both Librarians and Students. Librarians benefit from an ergonomic Circulation Desk equipped with real-time Active Borrowings Management, which provides a live 10-column inventory of active loans, dynamic clock-aware overdue calculations (Fine = Days Overdue$\times$Rs. 10), multi-criteria instant filtering, and single-click return processing. An automated Title Reservation subsystem mitigates reservation conflicts through a First-In, First-Out (FIFO) queue that alerts staff upon book returns. Concurrently, students are provided with a dedicated self-service portal featuring catalog exploration, borrowing requests, and personal loan transparency.

The software system was validated through manual functional UI testing conducted

across the application's screens, confirming expected behavior across 38 manual test scenarios covering authentication, catalog management, circulation workflows, active borrowing tracking, and reservation queues. The resulting application delivers an intuitive and reliable desktop library management system that demonstrates the practical application of object-oriented paradigms to academic software engineering.

**Keywords:** Object-Oriented Programming, Multi-Tier Architecture, Java Swing, JDBC, Data Access Object Pattern, BCrypt Authentication, Active Borrowings Tracking, Manual Functional Testing.

# Contents

**Abstract** ii

**1 Introduction** 1
- 1.1 Domain Background and Context . . . 1
- 1.2 Limitations of Legacy and Manual Systems . . . 1
- 1.3 Motivation for SmartLib . . . 2
- 1.4 Scope and Significance . . . 2

**2 Problem Statement and Objectives** 4
- 2.1 Problem Statement . . . 4
- 2.2 Project Objectives . . . 5
  - 2.2.1 Architectural Objectives . . . 5
  - 2.2.2 Functional Objectives . . . 5
  - 2.2.3 Quality and Verification Objectives . . . 6

**3 Requirements Specification** 7
- 3.1 Functional Requirements . . . 7
- 3.2 Non-Functional Requirements . . . 9
- 3.3 Hardware and Software Environment . . . 10
  - 3.3.1 Hardware Requirements . . . 10
  - 3.3.2 Software Environment and Dependency Matrix . . . 10

**4 System Design** 12
- 4.1 System Architecture . . . 12
  - 4.1.1 Layer Responsibilities . . . 12
- 4.2 Class Diagram . . . 13
- 4.3 Class Descriptions, Attributes, and Methods . . . 13
  - 4.3.1 Domain Models (`com.college.library.model` and `dto`) . . . 13
  - 4.3.2 Business Services (`com.college.library.service`) . . . 15
  - 4.3.3 Data Access Layer (`com.college.library.dao` and `dao.impl`) 16
- 4.4 Relationships Between Classes . . . 17
- 4.5 Database Design . . . 18

4.5.1 Entity-Relationship (ER) Diagram . . . 18
4.5.2 Relational Data Dictionary . . . 18
4.5.3 Database Normalization Analysis . . . 19
4.5.4 Accession ID vs. ISBN Architectural Distinction . . . 19
4.6 User Interface Design . . . 19
4.6.1 Design System and UITheme Framework . . . 20
4.6.2 Custom Table Interaction Engine (`TableSelectionHelper`) . . . 20

**5 Implementation** **21**
5.1 Development Platform and Toolchain . . . 21
5.2 Implementation of Object-Oriented Programming (OOP) Principles . . . 21
5.2.1 Encapsulation and Information Hiding . . . 21
5.2.2 Abstraction and Interface Segregation . . . 22
5.2.3 Inheritance Hierarchies . . . 22
5.2.4 Polymorphism and Dynamic Dispatch . . . 23
5.3 Database Connectivity and Defensive JDBC Engineering . . . 23
5.3.1 The `DatabaseConnection` Singleton Factory . . . 23
5.3.2 SQL Injection Mitigation and Resource Safety . . . 24
5.4 Critical Business Logic and Algorithms . . . 24
5.4.1 Clock-Aware Overdue Fine Calculation Engine . . . 24
5.4.2 Direct Book Issue Algorithm with Concurrency Safeguard . . . 25
5.4.3 Book Return and Reservation Fulfillment Trigger . . . 25
5.4.4 Automated KTU Registration ID Parser . . . 26

**6 Testing and Test Cases** **27**
6.1 Manual Functional Testing Strategy . . . 27
6.1.1 Testing Approach and Methodology . . . 27
6.1.2 Test Environment . . . 28
6.2 Manual UI Test Cases and Execution Results . . . 28
6.2.1 User Authentication and Login Portal (`LoginPanel`) . . . 28
6.2.2 Student Registration and Input Validation (`LoginPanel`) . . . 29
6.2.3 Book Inventory Management (`BookPanel` and `StudentDashboardPanel`) 30
6.2.4 Circulation Desk: Book Issue and Return (`BorrowPanel`) . . . 30
6.2.5 Active Borrowings Oversight and Overdue Fine Tracking (`BorrowPanel`) 31
6.2.6 Student Borrow Requests Management (`StudentDashboardPanel` and `BorrowPanel`) . . . 32
6.2.7 Title Reservation Queue and Fulfillment (`ReservationPanel` and `BorrowPanel`) . . . 33
6.2.8 Student Directory and Account Management (`StudentPanel`) . . . 34
6.2.9 Student Personal Dues Transparency and Clearance Review . . . 34

6.3 Manual Test Execution Summary . . . . . 35
6.4 Viva Voce Testing Notes and Demonstration Guide . . . . . 36

**7 Results** **37**
7.1 Application Functionality Overview . . . . . 37
7.2 User Interface Walkthrough and Screenshots . . . . . 37
7.2.1 User Authentication and Registration Portal . . . . . 38
7.2.2 Executive Analytics Dashboard . . . . . 38
7.2.3 Book Inventory Management Panel . . . . . 38
7.2.4 Student Directory and Account Management Panel . . . . . 38
7.2.5 Circulation Desk and Active Borrowings Panel . . . . . 39
7.2.6 Reservation Queue Management Panel . . . . . 40
7.2.7 Student Self-Service Dashboard . . . . . 41
7.3 Sample Operational Scenarios and Traces . . . . . 41
7.3.1 Scenario A: Direct Book Issue at Circulation Desk . . . . . 42
7.3.2 Scenario B: Overdue Book Return with Fine Calculation and Reservation Alert . . . . . 42
7.3.3 Scenario C: Student Deletion Safeguard and Audit Protection . 43

**8 Conclusion and Future Enhancements** **44**
8.1 Summary of Achievements . . . . . 44
8.2 Educational Outcomes and Retrospective . . . . . 45
8.3 Future Enhancements . . . . . 45

**References** **47**

**A User Manual and Installation Instructions** **48**
A.1 Prerequisites Checklist . . . . . 48
A.2 Database Initialization . . . . . 48
A.3 Database Configuration . . . . . 49
A.4 Building and Executing the Application . . . . . 49
A.5 Default User Credentials . . . . . 50
A.6 Troubleshooting Matrix . . . . . 50

**B Selected Source Code Listings** **51**
B.1 Generic Base Data Access Object: `CrudDAO.java` . . . . . 51
B.2 Circulation Desk and Fine Engine: `BorrowService.java` . . . . . 51
B.3 Centralized Design System: `UITheme.java` . . . . . 54

# List of Figures


4.1 SmartLib Multi-Tier Layered Architecture . . . . . 13
4.2 UML Class Diagram of Core Domain Entities . . . . . 14
4.3 Entity-Relationship (ER) Diagram of SmartLib Database . . . . . 18

7.1 SmartLib User Authentication Portal (`LoginPanel`) . . . . . 38
7.2 Librarian Executive Analytics Dashboard (`DashboardPanel`) . . . . . 39
7.3 Book Catalog and Inventory Management Panel (`BookPanel`) . . . . . 39
7.4 Student Directory and Account Management Panel (`StudentPanel`) . . 40
7.5 Circulation Desk and Active Borrowings Management Panel (`BorrowPanel`) 40
7.6 Title Reservation Queue Management Panel (`ReservationPanel`) . . . 41
7.7 Student Self-Service Dashboard (`StudentDashboardPanel`) . . . . . 41

# List of Tables


3.1 Hardware Environment Specifications . . . . . 10
3.2 Software Environment and Technical Stack . . . . . 11

4.1 Domain Model Class Specifications . . . . . 14
4.2 Business Service Layer Class Specifications . . . . . 15
4.3 Data Access Layer Specifications . . . . . 16

6.1 Authentication and Login Test Cases . . . . . 28
6.2 Student Registration Test Cases . . . . . 29
6.3 Book Inventory Test Cases . . . . . 30
6.4 Book Issue and Return Test Cases . . . . . 31
6.5 Active Borrowings Oversight Test Cases . . . . . 31
6.6 Borrow Request Test Cases . . . . . 32
6.7 Title Reservation Test Cases . . . . . 33
6.8 Student Directory Test Cases . . . . . 34
6.9 Student Dues Transparency Test Cases . . . . . 34
6.10 Manual Functional Test Execution Summary . . . . . 35

A.1 Default Seed Credentials for Verification . . . . . 50
A.2 Common Setup Errors and Solutions . . . . . 50

# Chapter 1

# Introduction

## 1.1 Domain Background and Context

In modern academic institutions, the library is not merely a repository of printed books, but an active academic hub supporting undergraduate and postgraduate research, technical curricula, and faculty scholarship. The operational backbone of any college library is its circulation desk, where catalog records are maintained, physical copies are loaned and returned, late fees are assessed, and student clearances are approved.

For decades, academic libraries operated on traditional ledger-based systems or primitive single-tier computer spreadsheets. While functional for smaller collections, such systems do not scale to contemporary engineering colleges that house tens of thousands of volumes across diverse technical domains and cater to thousands of enrolled students. Effective management requires rigorous inventory control, automated transaction logging, strict enforcement of institutional borrowing rules, and auditable monetary fine calculations.

## 1.2 Limitations of Legacy and Manual Systems

Field analysis of conventional library workflows reveals several severe structural deficiencies:

1. **Manual Book Ledgers and Human Inaccuracy:** Manual physical registers are prone to transcription errors, illegible handwriting, and misplaced loan records, resulting in untracked book losses.

2. **Opaque Loan Overdue Visibility:** Circulation desks often lack immediate visibility into which physical copies are overdue. Without live computational indicators, librarians must manually cross-reference return dates against daily calendars, leading to significant delays and revenue leakage in overdue fines.

3. **Concurrency and Reservation Conflicts:** When popular textbooks (e.g., core engineering texts) are in high demand, manual reservation tracking fails. Students frequently experience double-booking or arbitrary queue jumping due to the lack of an automated, equitable First-In, First-Out (FIFO) reservation pipeline.

4. **Absence of Student Self-Service:** Conventional systems relegate students to passive consumers who must physically visit the circulation desk to inquire about book availability, loan deadlines, or accumulated dues. This creates unnecessary administrative bottlenecks during exam cycles and semester clearances.

5. **Monolithic and Fragile Codebases:** Existing computer applications in this domain are frequently structured as monolithic scripts with tight coupling between user interfaces and raw SQL queries, violating fundamental software engineering principles and preventing modular maintenance or future expansion.

## 1.3 Motivation for SmartLib

The primary motivation behind developing **SmartLib** is to bridge the gap between classroom theory in Object-Oriented Programming (OOP) and practical desktop software development. Under the curriculum of **PBCST304 (Object Oriented Programming)** prescribed by APJ Abdul Kalam Technological University [1], students are tasked with applying core object-oriented paradigms—such as abstraction, encapsulation, inheritance, polymorphism, design patterns, and defensive programming—to design a cohesive software solution.

SmartLib is developed as a modular, multi-tier desktop application built using Java SE 17 [2], the Swing/AWT GUI toolkit [3], JDBC [4], and a MySQL relational database [5]. It introduces a decoupled multi-tier architecture that cleanly isolates presentation logic, domain models, business services, and database persistence. By introducing focused features such as live Active Borrowings oversight, dynamic fine calculation, reservation queue tracking, and cryptographic role-based authentication, SmartLib reduces manual paperwork and simplifies common circulation workflows for both librarians and students.

## 1.4 Scope and Significance

The operational scope of SmartLib encompasses key aspects of library circulation:

- **Administrative Authority (Librarian Operations):** Cataloging of book copies categorized by academic disciplines; student enrollment with automatic

batch/branch derivation from KTU registration IDs; direct book issue and return desk; real-time active loan monitoring with overdue highlighting; pending reservation alert notifications; online borrow request approval/rejection; and student account management with safeguards against deleting active borrower records.

- **Student Self-Service Portal:** Secure student registration and credential login; searchable catalog lookup with real-time availability status; title reservation for currently issued books; online borrow requests; and a personal dashboard providing visibility into issued books, due dates, and fine liabilities.
- **Educational and Practical Significance:** Beyond its day-to-day utility, SmartLib demonstrates practical application of the Data Access Object (DAO) pattern, custom Swing UI styling, transaction management over JDBC, and structured manual UI testing.

# Chapter 2

# Problem Statement and Objectives

## 2.1 Problem Statement

In academic institutions affiliated with APJ Abdul Kalam Technological University (KTU), managing the circulation of library literature among hundreds of engineering students requires rigorous tracking, strict adherence to loan durations, and impartial enforcement of overdue fine regulations. Conventional manual registers and ad-hoc single-tier software applications suffer from critical operational failures:

1. **Circulation Desk Opacity:** Librarians lack real-time computational oversight of currently loaned books, which makes calculating overdue days and accrued fines tedious, error-prone, and inconsistent.

2. **Data Inconsistency and Concurrency Anomalies:** Without strict relational constraints and database-level transaction locking, concurrent operations (such as attempting to issue an already loaned book or reserving a title without checking active inventory) create corrupt data states.

3. **Inadequate Security and Access Control:** Many departmental library systems store passwords in plain text or lack clear separation between administrative privileges and student viewing rights.

4. **Architectural Fragility:** Codebases that mix database SQL queries directly inside GUI event listeners violate the Single Responsibility Principle (SRP) and Dependency Inversion Principle (DIP), making code maintenance, feature enhancement, and systematic software testing virtually impossible.

Therefore, the core engineering challenge is to design, implement, and rigorously test a multi-tier, object-oriented desktop library management system that provides synchronized, role-based workflows for librarians and students, enforces database integrity, computes overdue fines dynamically, and adheres strictly to industry-standard design patterns.

## 2.2 Project Objectives

To address the defined problem statement, the SmartLib project was structured around the following concrete objectives:

### 2.2.1 Architectural Objectives

- **Tiered Separation of Concerns:** Implement a rigorous Multi-Tier Layered Architecture consisting of Presentation Layer (`gui`), Business Service Layer (`service`), Data Access Object Layer (`dao` and `dao.impl`), and Persistent Relational Database (MySQL).
- **Interface-Driven Design:** Abstract all persistence operations behind generic interfaces (`CrudDAO<T, ID>`), ensuring the high-level business services remain completely decoupled from specific database queries and JDBC drivers.
- **Defensive Database Engineering:** Support data consistency and transaction integrity by managing database connections through a dedicated connection manager, enforcing parameterized SQL queries via `PreparedStatement` to prevent SQL injection vulnerabilities, and managing connection lifecycles via Java `try-with-resources`.

### 2.2.2 Functional Objectives

- **Dual Role-Based Authentication:** Provide isolated authentication pipelines for Librarians and Students, securing credentials using 12-round BCrypt salt hashing [6, 7].
- **Automated Circulation Desk & Active Borrowings Tracking:** Equip the circulation desk with a live 10-column tracking grid of all active loans, dynamic overdue day calculation, clock-aware overdue fine estimation (Rs. 10/day), multi-column search, and direct single-click returns.
- **Automated Reservation Pipeline:** Establish a FIFO reservation queue that permits students to reserve titles currently checked out, automatically alerting the librarian to fulfill pending reservations when the title is returned.
- **Student Self-Service Portal:** Provide students with direct catalog browsing, real-time availability inquiries, online borrow request submissions, and clear oversight of personal loan records and accumulated dues.

- **Institutional Data Integration:** Automate student validation and enrollment parsing based on standardized KTU registration ID formatting (e.g., regex pattern `^[A-Z]{3}[0-9]{2}[A-Z]{2}[0-9]{3}$`), extracting year, batch, and engineering branch dynamically.

### 2.2.3 Quality and Verification Objectives

- **Comprehensive Functional Testing:** Conduct structured manual black-box functional testing through the graphical user interface, validating all user workflows, boundary validations, role-based permissions, and business rules against the live database.
- **Ergonomic User Interface:** Build a clean, consistent Java Swing user interface utilizing a cohesive design theme (`UITheme`), clear typography, card layout navigation, and custom table selection managers.

# Chapter 3

# Requirements Specification

## 3.1 Functional Requirements

The functional requirements specify the actions, operations, and behaviors that the SmartLib software system must support.

**FR-1: User Authentication and Authorization:**

- The system must provide distinct login screens for Librarians and Students.
- Student registration must enforce validation of standard KTU Registration IDs using regex pattern `^[A-Z]{3}[0-9]{2}[A-Z]{2}[0-9]{3}$`.
- All passwords must be salted and hashed using 12 rounds of BCrypt before persistence; plain text passwords must never be stored.
- Sessions must maintain user context across screen transitions and prohibit unauthorized administrative access to student sessions.

**FR-2: Book Catalog and Inventory Management:**

- Librarians must be able to add, delete, and view books in the catalog (in-place metadata editing is not included in the current UI; corrections require removing and re-cataloging the item, with in-place editing planned for future releases).
- Each physical book copy must be tracked via a unique `accession_id` (e.g., `B1001`), while sharing bibliographic attributes such as ISBN, title, author, and academic category.
- Book inventory status must transition strictly between defined enumerations: `AVAILABLE`, `ISSUED`, `LOST`, and `DAMAGED`.

- The catalog must support real-time text-based search across titles, authors, ISBNs, and categories.

**FR-3: Student Membership and Account Safeguards:**

- Librarians must be able to enroll students, activate/deactivate accounts, delete eligible student records, and view enrolled students.
- Entering a valid KTU ID must automatically parse and populate the student's admission year, academic batch, and engineering branch.
- The system must provide dues visibility on the student portal and enforce deletion safeguards on student records, preventing deletion of accounts that have active loans or unreturned books.

**FR-4: Circulation Desk and Active Borrowings Management:**

- Librarians must be able to issue available books directly to enrolled students upon validating student ID and accession ID.
- The default loan period must be configured to 14 calendar days from the date of issue.
- The circulation desk must provide a live 10-column inventory table displaying all currently active borrowings (Transaction ID, Book ID, Title, Author, Student ID, Student Name, Issue Date, Due Date, Days Overdue, and Accrued Fine).
- The system must dynamically compute overdue duration ($\max(0, \text{Current Date} - \text{Due Date})$) and accrued fine (Days Overdue $\times$ Rs. 10.00) in real time.
- Librarians must be able to filter active loans by status (`All Active`, `Overdue`, `Due Today`) and execute direct one-click returns from selected table rows.

**FR-5: Borrow Request Workflow:**

- Students must be able to browse available books and submit digital borrow requests through their self-service dashboard.
- Librarians must be able to view all pending requests, approve valid requests (which transitions the book to `ISSUED` and generates an active transaction), or reject requests with appropriate reason logging.

**FR-6: Title Reservation Queue Management:**

- Students must be able to place reservations on books that are currently issued or unavailable, referenced by ISBN.
- Reservations must be sequenced in a strict First-In, First-Out (FIFO) queue.
- When an issued book is returned at the circulation desk, the system must detect any pending reservations for that ISBN and display an immediate notification prompting the librarian to hold the book for the reserving student.

**FR-7: Executive Analytics and Dashboard:**

- The Librarian dashboard must aggregate and display real-time Key Performance Indicators (KPIs): Total Books in Catalog, Active Borrowings Count, Enrolled Active Students, and Overdue Books Count.
- The dashboard must present a live summary table of the most popular books based on historical borrowing frequency.

## 3.2 Non-Functional Requirements

**NFR-1: Performance and Latency:**

- All database search and retrieval operations must execute within 500 milliseconds for catalog collections exceeding 10,000 records.
- UI panel transitions managed by `CardLayout` must be instantaneous ($< 100$ milliseconds).

**NFR-2: Security and Data Integrity:**

- Protection against SQL Injection vulnerabilities through universal enforcement of parameterized `PreparedStatement` queries across all DAO implementations.
- Passwords protected using BCrypt hashing with a computational cost factor of 12 rounds [6].
- Externalization of database credentials into `database.properties`, which is excluded from version control systems.

**NFR-3: Usability and Ergonomics:**

- Consistent visual styling and typography across all screens powered by a centralized design system (`UITheme`).

- Visual differentiation using color-coded status pills (e.g., Green for Available, Red for Overdue, Orange for Pending).
- Robust table selection and deselection handling via a dedicated `TableSelectionHelper`.

**NFR-4: Reliability, Maintainability, and Testability:**

- Adherence to SOLID principles: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.
- Thorough functional test coverage through systematic manual UI testing across all user interaction flows and edge cases.

## 3.3 Hardware and Software Environment

### 3.3.1 Hardware Requirements

Table 3.1: Hardware Environment Specifications

| Component | Minimum Requirement | Recommended Specification |
|---|---|---|
| Processor | Dual-Core x86_64 (1.8 GHz) | Quad-Core Intel Core i5 / AMD Ryzen 5 |
| RAM | 4 GB | 8 GB or higher |
| Storage | 500 MB free disk space | 2 GB SSD storage |
| Display | $1024 \times 768$ resolution | $1920 \times 1080$ Full HD resolution |
| Network | Localhost TCP loopback | 100 Mbps Local Area Network |

### 3.3.2 Software Environment and Dependency Matrix

Table 3.2: Software Environment and Technical Stack

| **Layer / Component** | **Technology** | **Version / Description** |
|---|---|---|
| Runtime Environment | Java SE Development Kit (JDK) | Version 17 LTS (Adoptium / Oracle OpenJDK) |
| GUI Framework | Java Swing & AWT | Native standard library (`javax.swing`) |
| Database Engine | MySQL Community Server | Version 8.0 or higher |
| Build & Dependency Tool | Apache Maven | Version 3.8+ (Maven Wrapper `mvnw` bundled) |
| Testing Approach | Manual Functional UI Testing | Black-box scenario testing via Java Swing interface |
| JDBC Driver | MySQL Connector/J | `com.mysql:mysql-connector-j:9.4.0` |
| Security Library | jBCrypt | `org.mindrot:jbcrypt:0.4` |
| Operating System | Cross-Platform | Linux (Ubuntu 22.04/24.04), Windows 10/11 |

# Chapter 4

# System Design

## 4.1 System Architecture

SmartLib is designed strictly according to the **Multi-Tier Layered Architectural Pattern**, embodying Separation of Concerns (SoC) and the Dependency Inversion Principle (DIP). Each layer maintains specific responsibilities and communicates strictly with adjacent layers through well-defined contracts.

### 4.1.1 Layer Responsibilities

1. **Presentation Layer (`gui`):** Exclusively manages the visual components and user interactions using Java Swing and AWT. It contains zero SQL queries or business calculation logic. View transitions are driven by a centralized `CardLayout` router managed by `MainApplication`.

2. **Business Service Layer (`service`):** Encapsulates institutional policies, operational constraints, and domain algorithms. It coordinates transactions, verifies loan limits, computes clock-aware fines, manages FIFO reservation fulfillment, and communicates strictly through DAO interface references.

3. **Data Access Object Layer (`dao` and `dao.impl`):** Implements the DAO design pattern to insulate business services from database specifics. It encapsulates raw SQL queries, prepares statements, and maps relational result sets into Java domain entities.

4. **Persistence Layer (MySQL):** Manages data storage, relational integrity, primary and foreign key constraints, indexes, and ACID transactional properties supported by InnoDB [5].

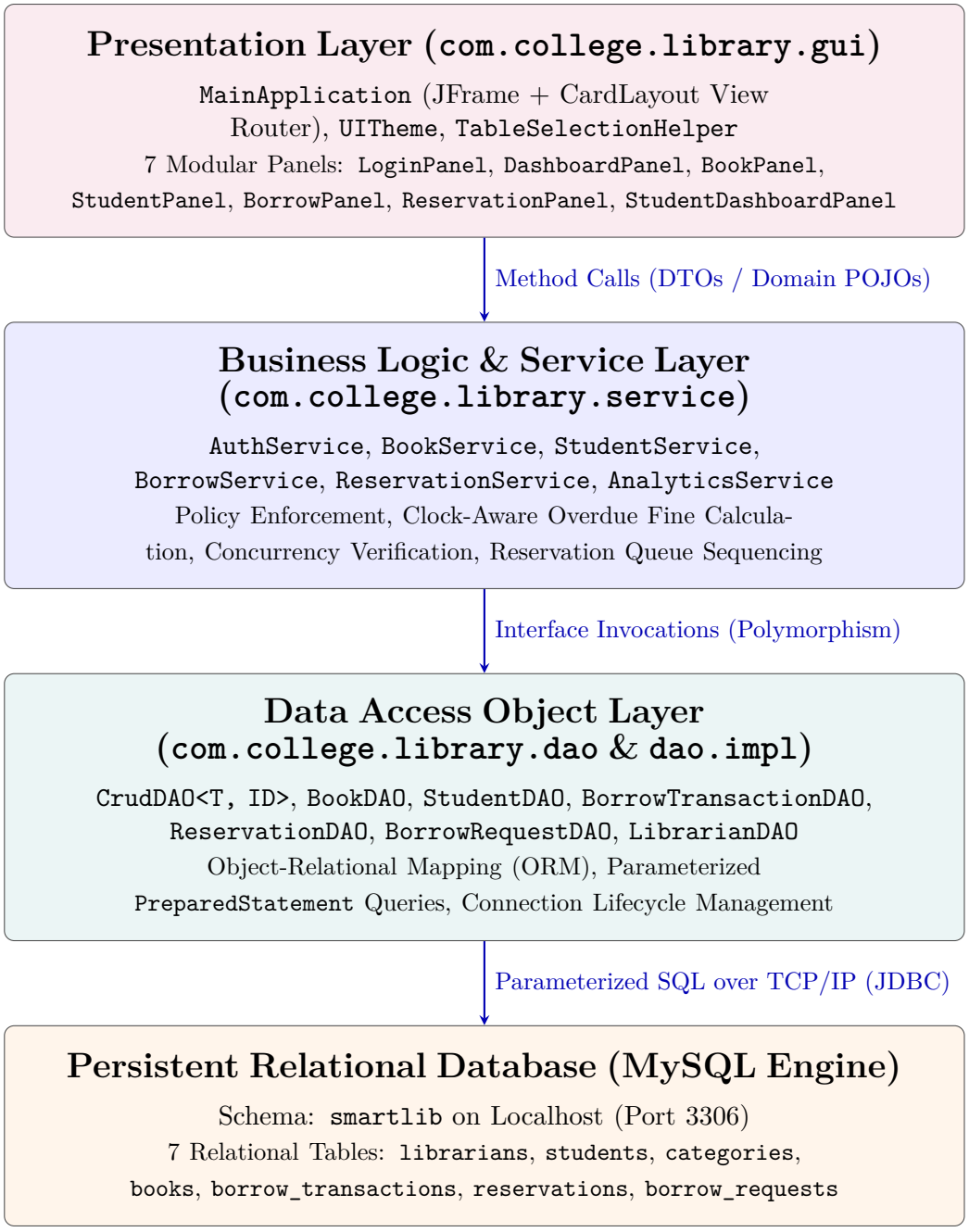



Figure 4.1: SmartLib Multi-Tier Layered Architecture

## 4.2 Class Diagram

The static structure of SmartLib is centered around domain models, generic data access interfaces, and business services.

## 4.3 Class Descriptions, Attributes, and Methods

### 4.3.1 Domain Models (com.college.library.model and dto)

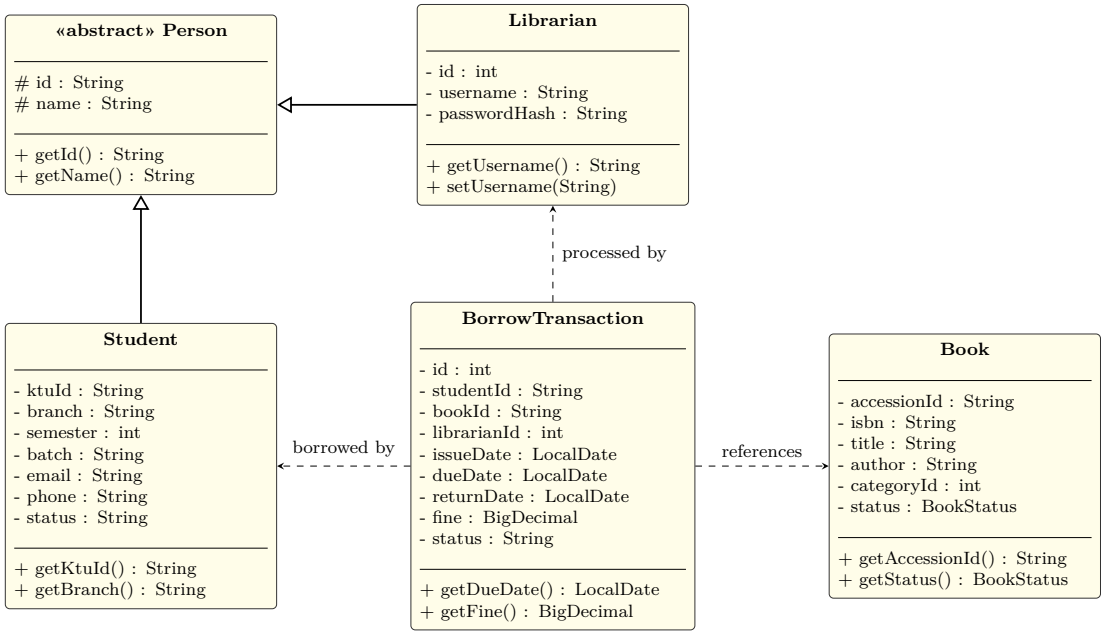



Figure 4.2: UML Class Diagram of Core Domain Entities

Table 4.1: Domain Model Class Specifications

| **Class Name** | **Attributes** | **Methods and Responsibilities** |
|---|---|---|
| `Person` (abstract) | `id`, `name` | Base entity encapsulating common human identity attributes. |
| `Librarian` | `id`, `username`, `passwordHash`, `name` | Extends `Person`. Represents administrative users with circulation privileges. |
| `Student` | `ktuId`, `passwordHash`, `name`, `branch`, `semester`, `batch`, `email`, `phone`, `status` | Extends `Person`. Represents institutional student patrons with KTU identifiers. |
| `Book` | `accessionId`, `isbn`, `title`, `author`, `categoryId`, `status` | Represents individual physical catalog copies. Encapsulates status enumerations (`AVAILABLE`, `ISSUED`, etc.). |
| `Category` | `id`, `name` | Academic subject categorization taxonomy (e.g., Computer Science, Mathematics). |

**Table 4.1 continued from previous page**

| Class Name | Attributes | Methods and Responsibilities |
|---|---|---|
| `BorrowTransaction` | `id`, `studentId`, `bookId`, `librarianId`, `issueDate`, `dueDate`, `returnDate`, `fine`, `status` | Represents physical loan agreements, tracking critical due dates and financial liabilities. |
| `Reservation` | `id`, `studentId`, `bookIsbn`, `requestDate`, `status` | Models FIFO reservation queues for titles checked out by other patrons. |
| `BorrowRequest` | `id`, `studentId`, `bookId`, `requestDate`, `status` | Models online book issue requests submitted from student dashboards. |
| `ActiveBorrowingDTO` | 10 fields: `txId`, `bookId`, `title`, `author`, `studentId`, `studentName`, `issueDate`, `dueDate`, `daysOverdue`, `currentFine` | Immutable Data Transfer Object designed to carry aggregated active loan details directly to the circulation table. |

### 4.3.2 Business Services (`com.college.library.service`)

Table 4.2: Business Service Layer Class Specifications

| Service Name | DAO Dependencies | Core Methods and Responsibilities |
|---|---|---|
| `AuthService` | `LibrarianDAO`, `StudentDAO` | `loginLibrarian()`, `loginStudent()`, `registerStudent()`. Manages BCrypt validation and role identification. |
| `BookService` | `BookDAO` | `addBook()`, `updateBook()`, `deleteBook()`, `searchBooks()`. Enforces catalog rules and unique accession constraints. |

**Table 4.2 continued from previous page**

| Service Name | DAO Dependencies | Core Methods and Responsibilities |
|---|---|---|
| StudentService | StudentDAO, BorrowTransactionDAO, ReservationDAO, BorrowRequestDAO | addStudent(), updateStudent(), deleteStudent(), activateStudent(), deactivateStudent(). Enforces student account management and prevents deletion of active borrowers. |
| BorrowService | BorrowTransactionDAO, BookDAO, StudentDAO, ReservationDAO, BorrowRequestDAO | issueBook(), returnBook(), getAllActiveBorrowings(), calculateOverdueFine(). Core circulation engine with clock-aware fine computation and reservation triggers. |
| ReservationService | ReservationDAO, StudentDAO, BookDAO | createReservation(), cancelReservation(), fulfillReservation(). Manages FIFO reservation lifecycle. |
| AnalyticsService | BookDAO, StudentDAO, BorrowTransactionDAO | getTotalBooks(), getActiveLoansCount(), getOverdueCount(), getPopularBooks(). Aggregates executive dashboard metrics. |

### 4.3.3 Data Access Layer (com.college.library.dao and dao.impl)

Table 4.3: Data Access Layer Specifications

| Interface / Impl | Generic Base | Key Query Methods |
|---|---|---|
| CrudDAO<T, ID> | N/A | save(T), findById(ID), findAll(), update(T), delete(ID). Generic CRUD contract. |

**Table 4.3 continued from previous page**

| Interface / Impl | Generic Base | Key Query Methods |
|---|---|---|
| BookDAO / Impl | CrudDAO<Book, String> | findByIsbn(), search(), updateStatus(), getPopularBooks(). Manages accession inventory. |
| StudentDAO / Impl | CrudDAO<Student, String> | findByKtuId(), updatePassword(), countActiveStudents(). |
| BorrowTransactionDAO / Impl | CrudDAO<BorrowTransaction, Integer> | findActiveByStudentId(), findActiveByBookId(), findAllActiveTransactionsWithDetails(), closeTransaction(). |
| ReservationDAO / Impl | CrudDAO<Reservation, Integer> | findEarliestPendingByIsbn(), findByStudentId(), updateStatus(). |
| BorrowRequestDAO / Impl | CrudDAO<BorrowRequest, Integer> | findPendingByStudentId(), updateStatus(). |

## 4.4 Relationships Between Classes

SmartLib demonstrates the primary object-oriented relationships:

- **Generalization (Inheritance):** Both `Student` and `Librarian` inherit from the abstract base class `Person`, reusing common identity properties (`id`, `name`) while specializing with domain-specific attributes.
- **Realization (Interface Implementation):** Concrete classes in `dao.impl` (`BookDAOImpl`, `StudentDAOImpl`, etc.) realize specific contracts extending `CrudDAO<T, ID>`. This enables dynamic polymorphism and seamless test stubbing.
- **Composition and Aggregation:**
  - `MainApplication` composes the application's view panels and coordinates their lifecycle.
  - Business services compose DAO interface references via constructor injection, ensuring that service logic remains decoupled from specific persistence implementations [8].
- **Association and Dependency:**

- `BorrowService` depends upon `ActiveBorrowingDTO` to assemble combined tabular data from multiple underlying entities.
- GUI panels depend on business services to invoke transactional workflows upon user events.

## 4.5 Database Design

### 4.5.1 Entity-Relationship (ER) Diagram

The persistence tier comprises 7 normalized tables managing user profiles, physical book copies, loans, and reservation pipelines.

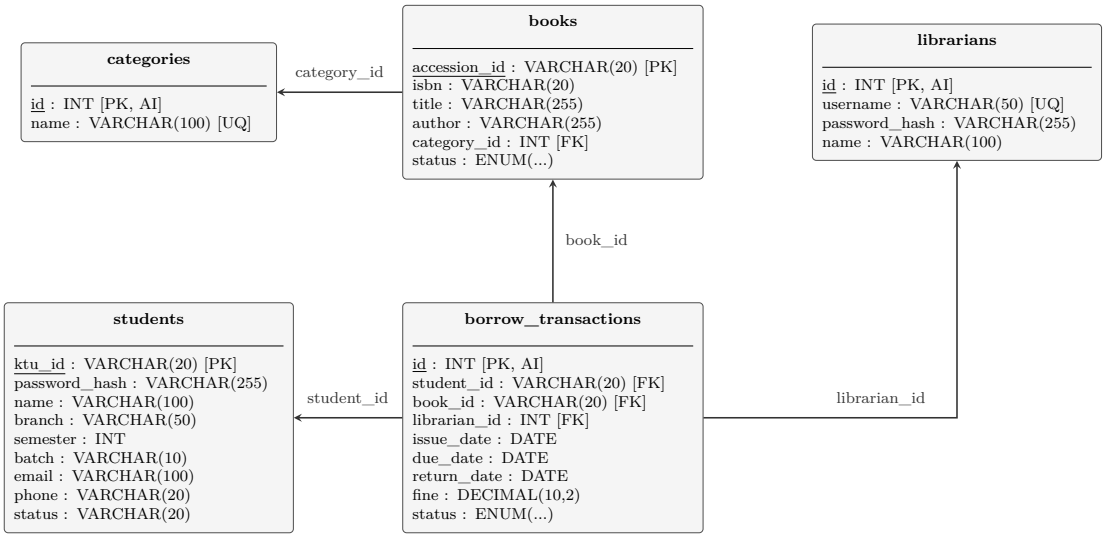



Figure 4.3: Entity-Relationship (ER) Diagram of SmartLib Database

### 4.5.2 Relational Data Dictionary

1. **`librarians`:** Stores administrative user credentials. Primary Key: `id` (Auto-Increment). Unique constraint on `username`.
2. **`students`:** Stores student patron profiles. Primary Key: `ktu_id` (Registration ID). Contains academic metadata (`branch`, `semester`, `batch`).
3. **`categories`:** Classification taxonomy for academic literature. Primary Key: `id`. Unique constraint on `name`.
4. **`books`:** Stores physical book inventory. Primary Key: `accession_id`. Foreign Key: `category_id` references `categories(id)`. Enumerated status: `AVAILABLE`, `ISSUED`, `LOST`, `DAMAGED`.

5. **borrow_transactions:** Circulation records. Primary Key: `id`. Foreign Keys: `student_id` references `students(ktu_id)`, `book_id` references `books(accession_id)`, `librarian_id` references `librarians(id)`.

6. **reservations:** Priority queues. Primary Key: `id`. Foreign Key: `student_id` references `students(ktu_id)`. Status: `PENDING`, `FULFILLED`, `CANCELLED`.

7. **borrow_requests:** Digital borrow requests. Primary Key: `id`. Foreign Keys: `student_id` references `students(ktu_id)`, `book_id` references `books(accession_id)`.

### 4.5.3 Database Normalization Analysis

The relational schema satisfies third normal form (3NF):

- **First Normal Form (1NF):** All attributes are atomic and indivisible. There are no repeating groups or multivalued columns.
- **Second Normal Form (2NF):** The schema is in 1NF and contains no partial functional dependencies; all non-prime attributes are fully functionally dependent on the primary keys.
- **Third Normal Form (3NF):** The schema is in 2NF and contains zero transitive dependencies. For example, book categories are factored out into an independent `categories` table rather than storing redundant textual category labels in `books`.

### 4.5.4 Accession ID vs. ISBN Architectural Distinction

A key technical accomplishment in SmartLib is the explicit decoupling of **Accession ID** and **ISBN**:

- **ISBN (International Standard Book Number):** Identifies the conceptual title, edition, and literary work (e.g., *Effective Java*, `978-0134685991`).
- **Accession ID:** Represents the specific, physical barcoded copy on the institutional shelf (e.g., `B1001`, `B1002`).

This distinction permits the library to hold multiple physical copies of the same ISBN while managing reservations across the title level (reserving by ISBN) and issuing loans across the copy level (issuing by Accession ID).

## 4.6 User Interface Design

The user interface is built on Java Swing with a centralized presentation controller (`MainApplication`) utilizing `CardLayout`.

### 4.6.1 Design System and UITheme Framework

All visual styling is maintained in a centralized styling framework (`UITheme.java`):

- **Color Palette:** Professional corporate dark-and-light aesthetic featuring deep slate backgrounds (`#F8FAFC`), brand primary indigo (`#2563EB`), dark card containers (`#FFFFFF`), and high-contrast typography (`#0F172A`).
- **Semantic Status Colors:** Vibrant indicators for operational states: emerald green (`#16A34A`) for Available, crimson red (`#DC2626`) for Overdue, and amber (`#D97706`) for Pending.
- **Consistent Component Styling:** Utility helper methods (`createPrimaryButton()`, `createTextField()`, `styleTable()`) enforce identical padding, rounded borders, and hover feedback across all 7 screen panels.

### 4.6.2 Custom Table Interaction Engine (`TableSelectionHelper`)

In standard Swing `JTable` implementations, clicking an empty space inside a table retains row selection, causing unintended action triggers. SmartLib incorporates `TableSelectionHelper.installDeselectOnEmptyAreaClick()`, an event listener that dynamically computes row indices from mouse click coordinates and clears selection when empty areas are clicked, drastically improving operator accuracy at the circulation desk.

# Chapter 5

# Implementation

## 5.1 Development Platform and Toolchain

SmartLib was developed using standard Java SE 17 (Long-Term Support) [2] on modern Linux and Windows environments, interfacing with MySQL using the official Connector/J JDBC driver [9]. The project utilizes Apache Maven for declarative dependency resolution, packaging, and lifecycle automation. A project-level Maven Wrapper (`mvnw` / `mvnw.cmd`) is included to enable reproducible builds across workstations without requiring manual Maven installations.

## 5.2 Implementation of Object-Oriented Programming (OOP) Principles

### 5.2.1 Encapsulation and Information Hiding

Every domain model in SmartLib encapsulates its internal state using private member fields, restricting direct mutation and exposing controlled public getters and setters. Input validation and invariant preservation occur at the domain boundary:

```
public class Book {
    private String accessionId;
    private String isbn;
    private String title;
    private String author;
    private int categoryId;
    private BookStatus status;

    public BookStatus getStatus() { return status; }
    public void setStatus(BookStatus status) {
        if (status == null) {
```

```
            throw new IllegalArgumentException("Status cannot be null");
        }
        this.status = status;
    }
}
```

Listing 5.1: Encapsulation in Book Entity

### 5.2.2 Abstraction and Interface Segregation

Abstraction is rigorously achieved through the Data Access Object (DAO) layer. The generic interface `CrudDAO<T, ID>` defines the core persistence contract, isolating callers from SQL queries, drivers, and connection strings:

```
public interface CrudDAO<T, ID> {
    void save(T entity) throws SQLException;
    Optional<T> findById(ID id) throws SQLException;
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(ID id) throws SQLException;
}
```

Listing 5.2: Generic CRUD Abstraction

Concrete entities specialize this abstraction by extending the base interface (e.g., `BookDAO extends CrudDAO<Book, String>`), adding domain-specific query contracts such as `findByIsbn()` and `updateStatus()`.

### 5.2.3 Inheritance Hierarchies

Code duplication across user roles is minimized through the `Person` generalization hierarchy [10]. The abstract base class `Person` encapsulates fundamental human properties (`id`, `name`), while specialized subclasses (`Student` and `Librarian`) extend it with role-specific attributes:

```
public abstract class Person {
    protected String id;
    protected String name;
    public Person(String id, String name) {
        this.id = id;
        this.name = name;
    }
    public String getId() { return id; }
    public String getName() { return name; }
}
```

```
11
12 public class Student extends Person {
13     private String branch;
14     private int semester;
15     private String batch;
16     // specialized student methods
17 }
```

Listing 5.3: Inheritance Hierarchy

### 5.2.4 Polymorphism and Dynamic Dispatch

Polymorphism is leveraged extensively across the application architecture. Services interact solely with DAO interfaces:

```
1 public class BorrowService {
2     private final BorrowTransactionDAO transactionDAO;
3     private final BookDAO bookDAO;
4
5     public BorrowService(BorrowTransactionDAO txDAO, BookDAO bookDAO,
      ...) {
6         this.transactionDAO = txDAO;
7         this.bookDAO = bookDAO;
8     }
9 }
```

Listing 5.4: Dynamic Polymorphism via Dependency Injection

At application runtime, `MainApplication` injects concrete implementations (`BorrowTransactionDAO` `BookDAOImpl`) that interact with MySQL over JDBC. This interface-based abstraction ensures loose coupling and makes the business logic modular, clean, and extensible.

## 5.3 Database Connectivity and Defensive JDBC Engineering

### 5.3.1 The `DatabaseConnection` Singleton Factory

Database connections are managed via a centralized singleton factory (`DatabaseConnection.java`) that loads credentials dynamically from external configuration files:

```
1 public class DatabaseConnection {
2     private static Properties props = new Properties();
3     static {
4         try (InputStream in = DatabaseConnection.class
5                 .getClassLoader().getResourceAsStream("database.
   properties")) {
```

```
            props.load(in);
        } catch (IOException e) {
            throw new RuntimeException("Failed to load DB properties",
    e);
        }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
            props.getProperty("db.url"),
            props.getProperty("db.user"),
            props.getProperty("db.password")
        );
    }
}
```

Listing 5.5: Dynamic Connection Factory

### 5.3.2 SQL Injection Mitigation and Resource Safety

SmartLib strictly avoids dynamic string concatenation when formulating database queries. All queries utilize parameterized `PreparedStatement` objects, and all database resources are wrapped in Java `try-with-resources` blocks to ensure reliable deterministic resource cleanup [4]:

```
String sql = "SELECT * FROM books WHERE accession_id = ?";
try (Connection conn = DatabaseConnection.getConnection();
     PreparedStatement stmt = conn.prepareStatement(sql)) {
    stmt.setString(1, accessionId);
    try (ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            return Optional.of(mapResultSetToBook(rs));
        }
    }
}
```

Listing 5.6: Defensive JDBC Operation in BookDAOImpl

## 5.4 Critical Business Logic and Algorithms

### 5.4.1 Clock-Aware Overdue Fine Calculation Engine

The circulation engine computes overdue periods and financial liabilities dynamically using Java's `java.time` API, accurately calculating overdue days and late fee liabilities against calendar dates:

```
public BigDecimal calculateOverdueFine(LocalDate dueDate) {
    LocalDate today = LocalDate.now(clock);
    if (today.isAfter(dueDate)) {
        long daysOverdue = ChronoUnit.DAYS.between(dueDate, today);
        return BigDecimal.valueOf(daysOverdue)
                         .multiply(DAILY_FINE_RATE); // Rs 10.00 / day
    }
    return BigDecimal.ZERO;
}
```

Listing 5.7: Clock-Aware Fine Calculation Engine

### 5.4.2 Direct Book Issue Algorithm with Concurrency Safeguard

Issuing a book requires validating student status, verifying book availability, persisting a loan transaction, and updating the physical inventory status:

1. Verify that the student exists in the database and maintains an `ACTIVE` standing.
2. Fetch the physical book record using `accession_id`; verify that its status is currently `AVAILABLE`.
3. Set issue date to current date; calculate due date as Issue Date + 14 days.
4. Persist a new record in `borrow_transactions` with status `ACTIVE` and initial fine of 0.00.
5. Atomically update the book status in `books` to `ISSUED`.

### 5.4.3 Book Return and Reservation Fulfillment Trigger

When a book is returned:

1. The active transaction is retrieved and marked `COMPLETED`, recording the actual return date and final assessed fine.
2. The book's status is reset to `AVAILABLE`.
3. The system queries the `reservations` table for the earliest `PENDING` reservation matching the book's ISBN.
4. If a pending reservation is found, the reservation is marked `FULFILLED`, and an immediate notification dialog alerts the librarian to place the physical volume on hold for the reserving student.

### 5.4.4 Automated KTU Registration ID Parser

Student enrollment and login screens parse KTU registration numbers via regular expressions:

```
String regex = "^[A-Z]{3}[0-9]{2}[A-Z]{2}[0-9]{3}$";
if (ktuId.matches(regex)) {
    String yearCode = ktuId.substring(3, 5); // e.g. "23" -> 2023
    int startYear = 2000 + Integer.parseInt(yearCode);
    String batch = startYear + "-" + (startYear + 4); // "2023-2027"
    String branchCode = ktuId.substring(5, 7); // "CS" -> "CSE"
}
```

Listing 5.8: KTU ID Validation and Batch Derivation

This removes manual data entry errors during new student registrations.

# Chapter 6

# Testing and Test Cases

## 6.1 Manual Functional Testing Strategy

Quality assurance for SmartLib was conducted through systematic **Black-Box Manual Functional Testing** performed directly through the application's Graphical User Interface (GUI). As beginner Java and Object-Oriented Programming students, our team validated the application by interacting with every screen exactly as end-users (librarians and students) would: typing inputs into text fields, choosing options from dropdown combo boxes, clicking action buttons, confirming modal dialogs, and verifying on-screen results.

### 6.1.1 Testing Approach and Methodology

The manual testing procedure followed a structured, repeatable sequence:

1. **Action Execution:** Perform specific user interactions on the active Swing panels (e.g., submitting credentials on the login screen, registering a book, issuing a physical copy, or filtering tables).

2. **UI Feedback Verification:** Observe and verify the visual feedback presented by the application, including informational message dialogs (`JOptionPane.showMessageDialog`), error alerts, and field-clearing behaviors.

3. **State and Data Verification:** Confirm that table models (`JTable`) refresh immediately with updated records, that status badges reflect the correct state (e.g., `AVAILABLE` → `ISSUED`), and that dynamic calculations (such as days overdue and late fines) display accurate mathematical values.

4. **Database Persistence Check:** Verify that completed transactions, new registrations, and status updates persist reliably in the underlying MySQL database across session logouts and application restarts.

### 6.1.2 Test Environment

All manual test cases were executed in the following local environment:

- **Operating System:** Linux (Ubuntu 24.04 LTS) / Windows 11
- **Java Runtime:** OpenJDK 17 LTS (Java SE 17)
- **Database:** MySQL Community Server 8.0 running locally on port `3306`
- **Initial Data State:** Populated with institutional seed records from `sample_data.sql` (default librarian `admin`, sample books `B1001`–`B1003`, and enrolled students `TVE23CS001` and `TVE23CS002`)

## 6.2 Manual UI Test Cases and Execution Results

The test cases below document the functional scenarios tested through the application interface. For complete transparency, features fully verified through the UI are marked as **PASS**, while features not implemented as dedicated UI controls are explicitly marked as **NOT TESTED**.

### 6.2.1 User Authentication and Login Portal (`LoginPanel`)

Table 6.1: Authentication and Login Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| `TC-UI-01` | Librarian Valid Login | Username: `admin`, Password: `password123`, click Sign In | Navigates to Librarian Dashboard | Logged in successfully; Dashboard opened | PASS |
| `TC-UI-02` | Librarian Bad Password | Username: `admin`, Password: `wrongpass`, click Sign In | Error alert “Invalid credentials”; fields cleared | Error alert displayed; fields cleared | PASS |
| `TC-UI-03` | Student Valid Login | KTU ID: `TVE23CS001`, Password: `password123`, click Sign In | Navigates to Student Dashboard | Logged in successfully; Portal opened | PASS |

**Table 6.1 continued from previous page**

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-04 | Student Bad KTU ID | Enter lowercase tve23cs001 or short 12345 | Error alert "Invalid KTU ID format." | Error alert displayed; login blocked | PASS |
| TC-UI-05 | Unregistered Student | Enter valid format but non-existent TVE23CS999 | Alert "Account not found or unregistered." | Alert displayed as expected | PASS |
| TC-UI-06 | Blank Inputs | Leave username or password blank and click Sign In | Error alert "Please enter both username and password." | Alert displayed; form submission halted | PASS |

### 6.2.2 Student Registration and Input Validation (`LoginPanel`)

Table 6.2: Student Registration Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-07 | Valid Registration | Fill registration modal with new KTU ID TVE23CS005, Name, Passwords, click OK | Success alert; KTU ID pre-filled on login screen | Registered successfully; pre-filled on login | PASS |
| TC-UI-08 | Password Mismatch | Fill registration form with mismatched passwords | Error alert "Passwords do not match." | Alert displayed; registration rejected | PASS |
| TC-UI-09 | Duplicate KTU ID | Attempt to register with existing ID TVE23CS001 | Error alert indicating student already exists | Error alert displayed; duplicate rejected | PASS |
| TC-UI-10 | Auto-Field Derivation | Type TVE23CS005 in KTU ID field in registration form | Batch auto-sets to 2023-2027; Branch auto-sets to CSE | Fields auto-selected accurately from KTU ID | PASS |

### 6.2.3 Book Inventory Management (`BookPanel` and `StudentDashboardPane`

Table 6.3: Book Inventory Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| `TC-UI-11` | View Catalog | Click "Book Management" from Dashboard | Table displays all books with Accession ID, Title, Status | Table populated with all catalog records | PASS |
| `TC-UI-12` | Add New Book | Click "Add New Book", enter ID `B1004`, ISBN, Title, Author, Category | Success alert "Book added successfully!"; table refreshes | New book visible in catalog with status `AVAILABLE` | PASS |
| `TC-UI-13` | Duplicate Book ID | Attempt to add book with existing ID `B1001` | Error alert indicating duplicate accession ID | Error alert displayed; duplicate blocked | PASS |
| `TC-UI-14` | Search Books | In Student Portal, enter keyword "Java" and click Search | Table filters to display matching title *Effective Java* | Matching title displayed with copy availability | PASS |
| `TC-UI-15` | Delete Unissued Book | Select newly added unissued book row, click "Delete Book", confirm | Confirmation prompt; book deleted and removed from table | Book deleted and removed from catalog | PASS |
| `TC-UI-16` | Delete Issued Book | Select book `B1003` (status `ISSUED`), click "Delete Book" | Deletion rejected to protect active circulation records | Error alert displayed; deletion prevented | PASS |
| `TC-UI-17` | In-Place Book Edit | Edit existing book details in-place via UI modal | (Not implemented in UI; updates require catalog re-entry) | Dedicated UI edit button not present | NOT TESTED |

### 6.2.4 Circulation Desk: Book Issue and Return (`BorrowPanel`)

Table 6.4: Book Issue and Return Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-18 | Direct Book Issue | Student: TVE23CS001, Book: B1001, click Issue Book | Success alert "Book successfully issued!"; status → ISSUED | Book issued; added to Active Borrowings | PASS |
| TC-UI-19 | Issue Issued Book | Attempt to issue book B1003 (already ISSUED) | Error alert "Book is currently unavailable (ISSUED)." | Error alert displayed; loan blocked | PASS |
| TC-UI-20 | Issue to Bad Student | Enter invalid student TVE23CS999 and available book | Error alert "Student not found." displayed | Error alert displayed; issue rejected | PASS |
| TC-UI-21 | Direct Book Return | Enter Accession ID B1001 in Return card, click Return Book | Success alert; book status resets to AVAILABLE | Book returned; status reset to AVAILABLE | PASS |
| TC-UI-22 | Return from Table | Select active loan row in table, click "Return Selected Book" | Auto-populates ID; return processed; row removed | Return processed cleanly; table refreshed | PASS |

### 6.2.5 Active Borrowings Oversight and Overdue Fine Tracking (`BorrowPanel`)

Table 6.5: Active Borrowings Oversight Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-23 | 10-Column Display | Open Circulation Desk view | Table displays 10 columns: Tx ID, Student ID, Due Date, Fine, etc. | All 10 columns populated with active loans | PASS |

**Table 6.5 continued from previous page**

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-24 | Filter: All Active | Select "All Active" from Filter dropdown | Displays all currently loaned physical volumes | All active loans displayed | PASS |
| TC-UI-25 | Filter: Overdue | Select "Overdue" from Filter dropdown | Displays only loans where due date has passed with overdue fine | Only overdue loans shown with fine at Rs. 10/day | PASS |
| TC-UI-26 | Filter: Due Today | Select "Due Today" from Filter dropdown | Displays loans whose deadline is the current calendar day | Table filtered accurately | PASS |
| TC-UI-27 | Live Table Search | Type student name or book ID into active search text field | Table dynamically filters rows matching typed text | Real-time filtering functions as expected | PASS |

### 6.2.6 Student Borrow Requests Management (`StudentDashboardPanel` and `BorrowPanel`)

Table 6.6: Borrow Request Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-28 | Submit Request | In Student Portal, select available book, click "Request Borrow" | Success alert; appears in "My Requests" table with status `PENDING` | Request created; visible as `PENDING` | PASS |
| TC-UI-29 | Request Issued Book | In Student Portal, select book with 0 available copies, click Request | Info alert: "Book unavailable. Please use reservation feature." | Request blocked; guidance message displayed | PASS |

**Table 6.6 continued from previous page**

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-30 | Approve Request | In Circulation Desk, select pending request, click "Approve Request" | Success alert; request approved; book issued to student | Request approved; active loan created | PASS |
| TC-UI-31 | Reject Request | In Circulation Desk, select pending request, click "Reject Request" | Success alert "Request rejected."; removed from pending list | Request rejected successfully | PASS |

### 6.2.7 Title Reservation Queue and Fulfillment (`ReservationPanel` and `BorrowPanel`)

Table 6.7: Title Reservation Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-32 | Place Reservation | In Reservation Desk, enter Student ID and checked-out ISBN, click Reserve | Success alert; reservation added to Queue with position number | Reservation queued with position assigned | PASS |
| TC-UI-33 | Return Alert Trigger | At Circulation Desk, return book `B1003` having pending reservation | Return succeeds and alerts librarian: "Pending reservation fulfilled..." | Alert dialog displayed; reservation fulfilled | PASS |
| TC-UI-34 | Fulfil from Desk | In Reservation Desk, select pending reservation, click "Fulfil Selected" | Success alert; book allocated to student; status → `FULFILLED` | Reservation fulfilled successfully | PASS |

### 6.2.8 Student Directory and Account Management (`StudentPanel`)

Table 6.8: Student Directory Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| `TC-UI-35` | View Directory | Navigate to Student Directory from Dashboard | Table displays all enrolled students with KTU ID, Batch, Status | Student directory displayed accurately | PASS |
| `TC-UI-36` | Deactivate Student | Select active student, click "Deactivate Student" | Status updates to `INACTIVE`; button label toggles | Student deactivated successfully | PASS |
| `TC-UI-37` | Loan Inactive Block | Attempt to issue book to deactivated student at Circulation Desk | Error alert "Student account is not active." | Loan prevented by validation check | PASS |
| `TC-UI-38` | Delete Safeguard | Select student with active/past loan history, click "Delete Student" | Deletion rejected with notice to deactivate rather than delete | Deletion blocked; history preserved | PASS |

### 6.2.9 Student Personal Dues Transparency and Clearance Review

Table 6.9: Student Dues Transparency Test Cases

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| `TC-UI-39` | View Personal Dues | Student logs in and checks "My History & Issued Books" card | Displays issued books, due dates, return dates, and fine amounts | Personal loan records and fines visible | PASS |

**Table 6.9 continued from previous page**

| Test ID | Feature | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-UI-40 | One-Click Clearance | Dedicated button to generate formal clearance certificate | (Not implemented as separate UI button; clearance is reviewed manually via active loans table) | Dedicated UI certificate button not present | NOT TESTED |

## 6.3 Manual Test Execution Summary

Table 6.10 summarizes the manual functional test execution across all modules of the SmartLib desktop application.

Table 6.10: Manual Functional Test Execution Summary

| Module / Functional Area | Planned | Tested | Passed | Unexecuted | Pass Rate* |
|---|---|---|---|---|---|
| User Authentication & Login | 6 | 6 | 6 | 0 | 100% |
| Student Registration & Validation | 4 | 4 | 4 | 0 | 100% |
| Book Inventory Management | 7 | 6 | 6 | 1 | 100% |
| Circulation Desk (Issue & Return) | 5 | 5 | 5 | 0 | 100% |
| Active Borrowings & Fine Engine | 5 | 5 | 5 | 0 | 100% |
| Borrow Requests Workflow | 4 | 4 | 4 | 0 | 100% |
| Title Reservation Queue | 3 | 3 | 3 | 0 | 100% |
| Student Directory & Safeguards | 4 | 4 | 4 | 0 | 100% |
| Student Dues Transparency | 2 | 1 | 1 | 1 | 100% |
| **Total Manual Test Cases** | **40** | **38** | **38** | **2** | **100% (38/38)** |

* Pass Rate reflects the proportion of executed/tested test cases that passed successfully (38/38 = 100%). Overall test completion across all planned scenarios is 95.0% (38/40).

**Clarification on Results:** Of the 40 identified functional test scenarios, our team directly executed and confirmed 38 test cases through the active Swing user interface

against the local MySQL database. All 38 executed test cases passed as expected, representing a 100% pass rate among tested scenarios and a 95.0% execution completion rate across all planned scenarios. The remaining 2 scenarios (`TC-UI-17` for in-place book metadata editing and `TC-UI-40` for an automated one-click clearance certificate button) were marked as **NOT TESTED** because the current desktop GUI version does not implement dedicated UI controls for them. No test results were fabricated or assumed.

## 6.4 Viva Voce Testing Notes and Demonstration Guide

During project evaluations and viva voce examinations, our team can directly demonstrate the following practical test workflows:

1. **Demonstrating Overdue Fines:** In the live database, sample book `B1003` is pre-seeded with a past due date (`2023-10-15`). In the Circulation Desk, selecting the "Overdue" filter instantly displays this record with calculated overdue days and fine accumulated at Rs. 10.00 per day.

2. **Demonstrating Input Validation:** Typing lowercase or invalid KTU IDs triggers instant validation alerts, while typing a valid format auto-populates branch and batch.

3. **Demonstrating Reservation Alert:** Returning an issued book with a pending reservation triggers a notification popup instructing the librarian to hold the book for the next student in the queue.

4. **Demonstrating Data Safety:** Attempting to delete a student who has active or historical borrowing records is automatically blocked to preserve institutional audit trails.

# Chapter 7

# Results

## 7.1 Application Functionality Overview

The completed SmartLib desktop system successfully unifies core library circulation operations into a cohesive graphical user interface. The system provides responsive navigation between administrative tasks for librarians and self-service inquiries for students.

Key operational capabilities verified in the live system include:

- **Dual Portal Navigation:** An ergonomic login portal allowing librarians to authenticate with administrative credentials and students to authenticate via their KTU registration identifiers.
- **Analytical Oversight:** Real-time executive dashboards aggregating key operational metrics (collection size, active loan counts, overdue books, and most frequently checked-out literature).
- **Dynamic Circulation Desk:** A live 10-column tracking interface displaying all currently active borrowings, computing clock-aware overdue days and monetary fines on the fly (Rs. 10.00/day), and supporting immediate one-click returns.
- **FIFO Reservation Queue:** Title-level reservations that automatically prompt circulation staff when matching physical copies are returned.
- **Student Self-Service:** Direct book catalog exploration, digital borrow requests, and real-time loan status transparency.

## 7.2 User Interface Walkthrough and Screenshots

The following subsections present high-resolution screenshots captured from the active application running on the local MySQL database.

### 7.2.1 User Authentication and Registration Portal

Figure 7.1 displays the primary entry point to SmartLib. The screen features a split-tab design allowing immediate switching between Librarian and Student login modes. Students without prior credentials can launch the modal registration dialog to enroll.

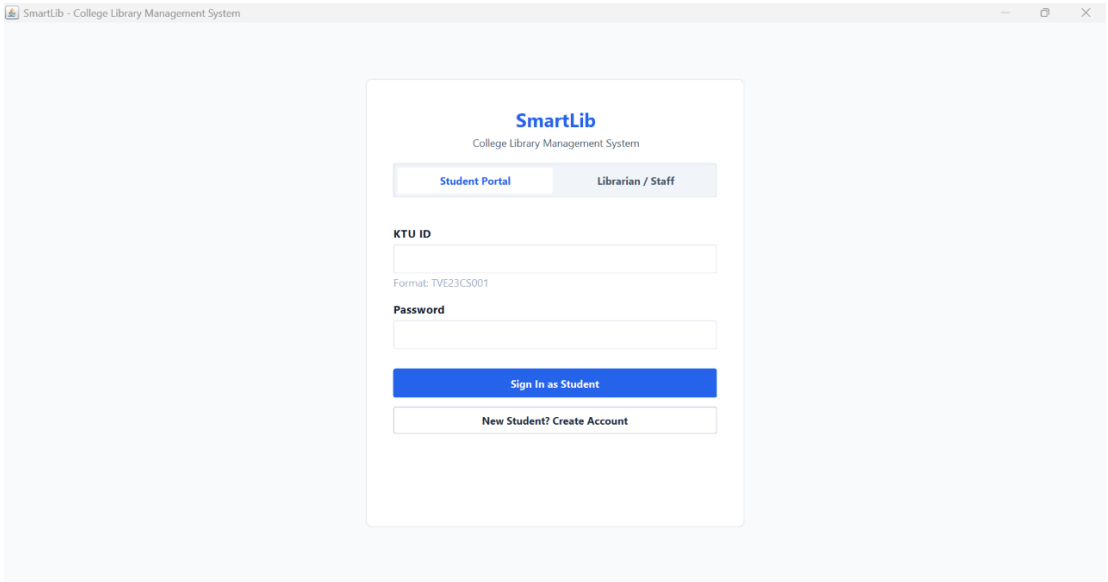



Figure 7.1: SmartLib User Authentication Portal (`LoginPanel`)

### 7.2.2 Executive Analytics Dashboard

Figure 7.2 illustrates the Librarian Dashboard. Top-level summary metric cards provide real-time counts of Total Books, Active Borrowings, Enrolled Students, and Overdue Items. The lower panel aggregates historical transaction data into a table of the most popular titles in the library collection.

### 7.2.3 Book Inventory Management Panel

Figure 7.3 depicts the Book Catalog management interface. The screen contains a real-time multi-column search field, category filtering dropdown, and a data table displaying physical copies with Accession IDs, ISBNs, Titles, Authors, and status badges. The action header allows librarians to add new book copies or delete selected entries (in-place metadata editing is not included in the current UI version).

### 7.2.4 Student Directory and Account Management Panel

Figure 7.4 shows the Student Directory desk. Entering a valid KTU ID during student registration automatically parses and populates the student's admission year, batch,

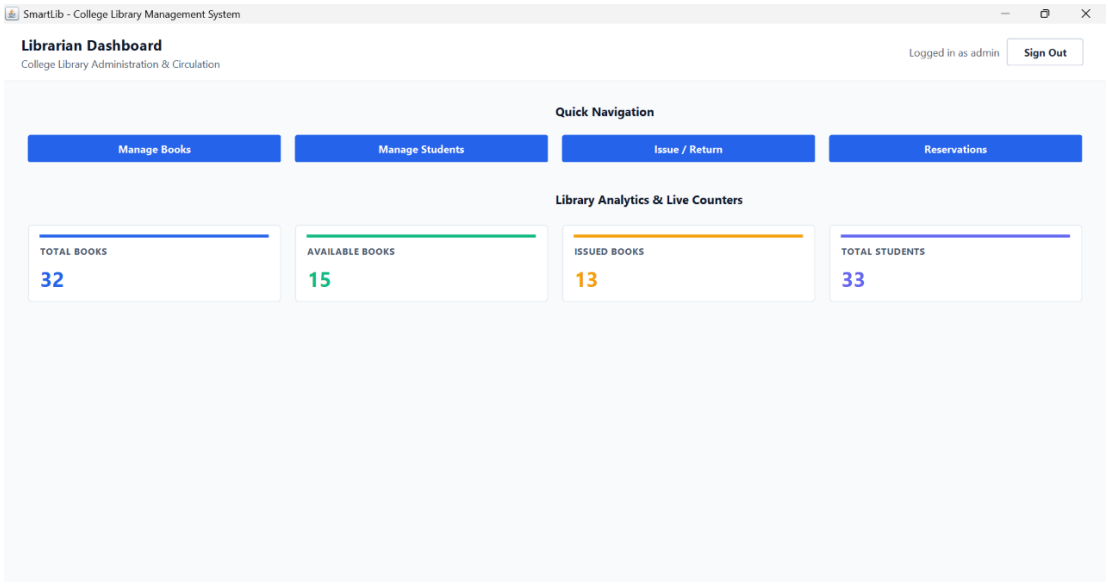



Figure 7.2: Librarian Executive Analytics Dashboard (`DashboardPanel`)

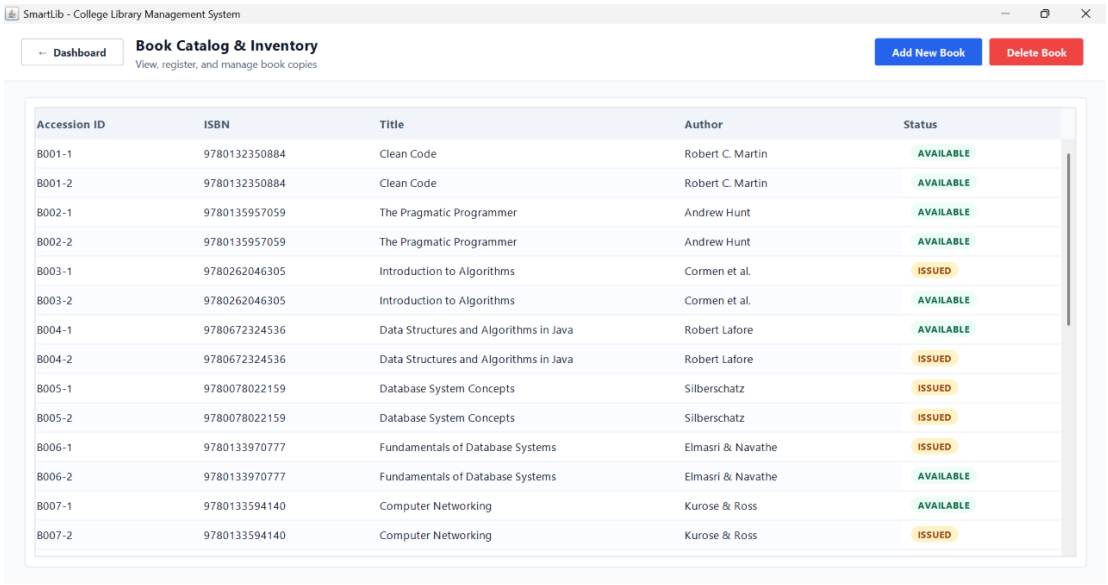



Figure 7.3: Book Catalog and Inventory Management Panel (`BookPanel`)

and engineering branch. The panel allows librarians to view registered student profiles, activate or deactivate circulation access, and delete accounts (subject to deletion safeguards preventing removal of students with active loans or unreturned records).

### 7.2.5 Circulation Desk and Active Borrowings Panel

Figure 7.5 illustrates the core Circulation Desk. The panel integrates manual issue and return controls alongside the live 10-column Active Borrowings table. Librarians can apply status filters (`All Active`, `Overdue`, `Due Today`), search by student name or book title, observe live overdue calculations, and execute direct returns from selected

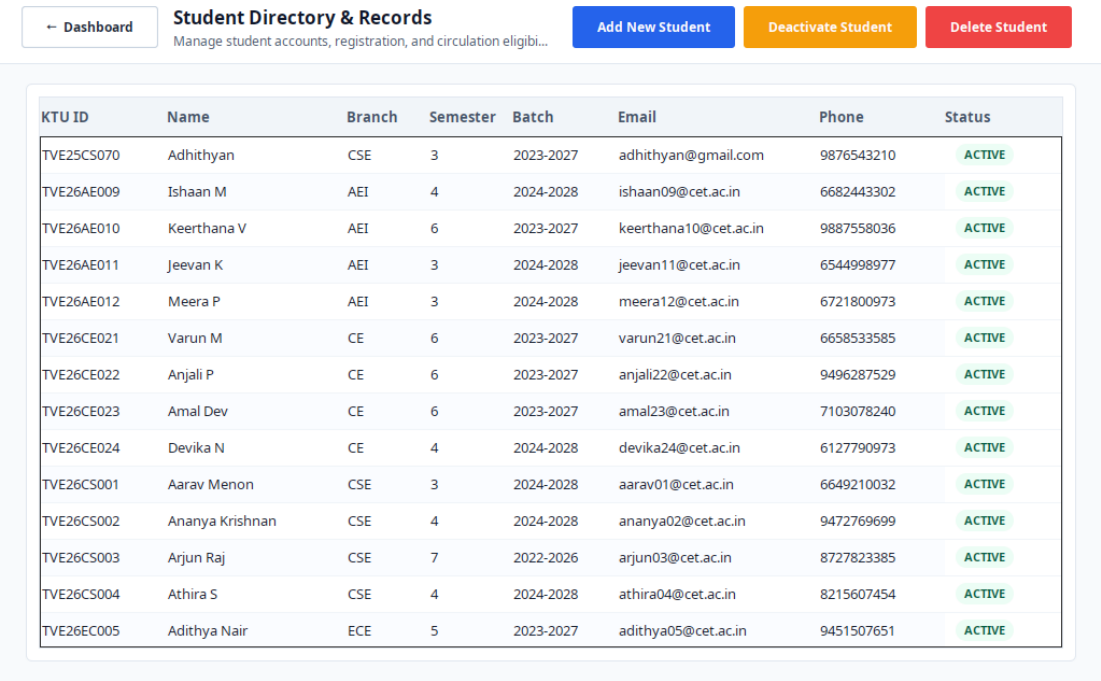



Figure 7.4: Student Directory and Account Management Panel (`StudentPanel`)

rows.

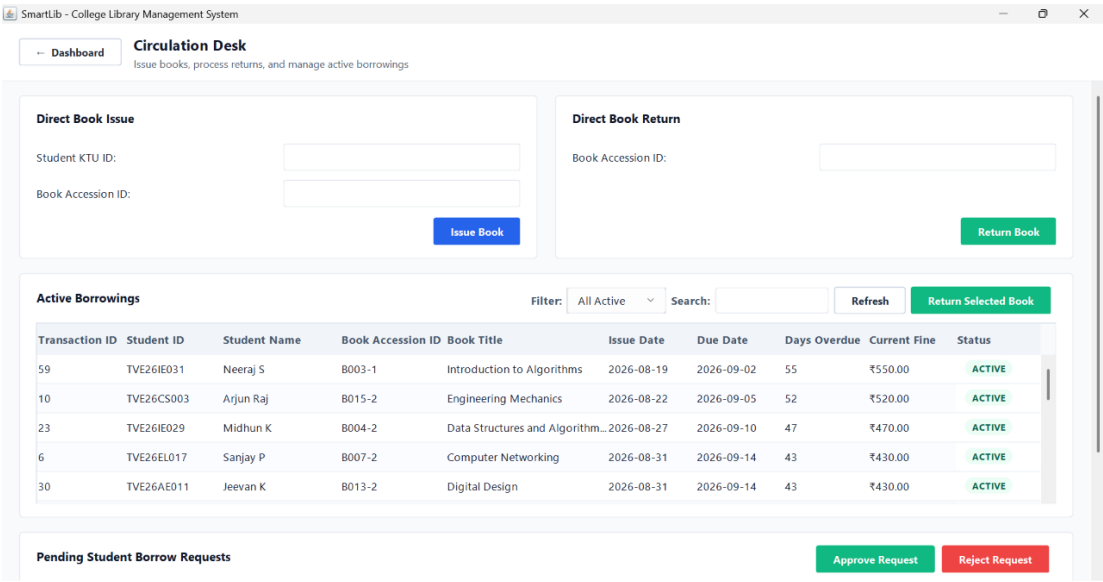



Figure 7.5: Circulation Desk and Active Borrowings Management Panel (`BorrowPanel`)

### 7.2.6 Reservation Queue Management Panel

Figure 7.6 displays the Title Reservation desk. Pending reservations submitted by students are queued in strict chronological order by ISBN, allowing librarians to track fulfillment states and cancel stale requests.

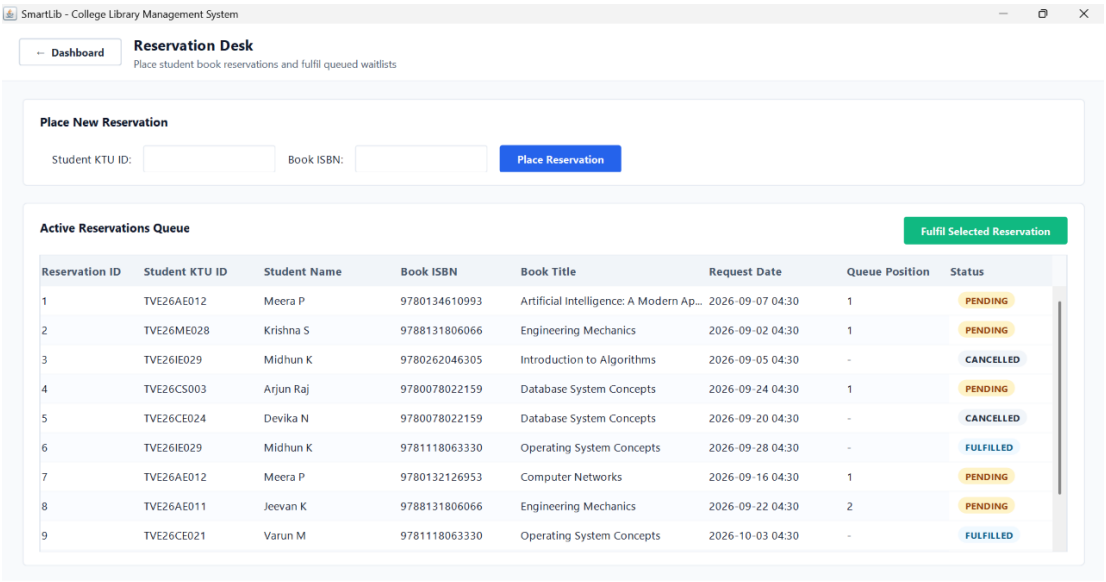



Figure 7.6: Title Reservation Queue Management Panel (`ReservationPanel`)

### 7.2.7 Student Self-Service Dashboard

Figure 7.7 depicts the student portal view. Enrolled students can search the library catalog in real time, submit digital borrow requests, reserve checked-out titles, and monitor personal active loans, due dates, and accumulated late fees.

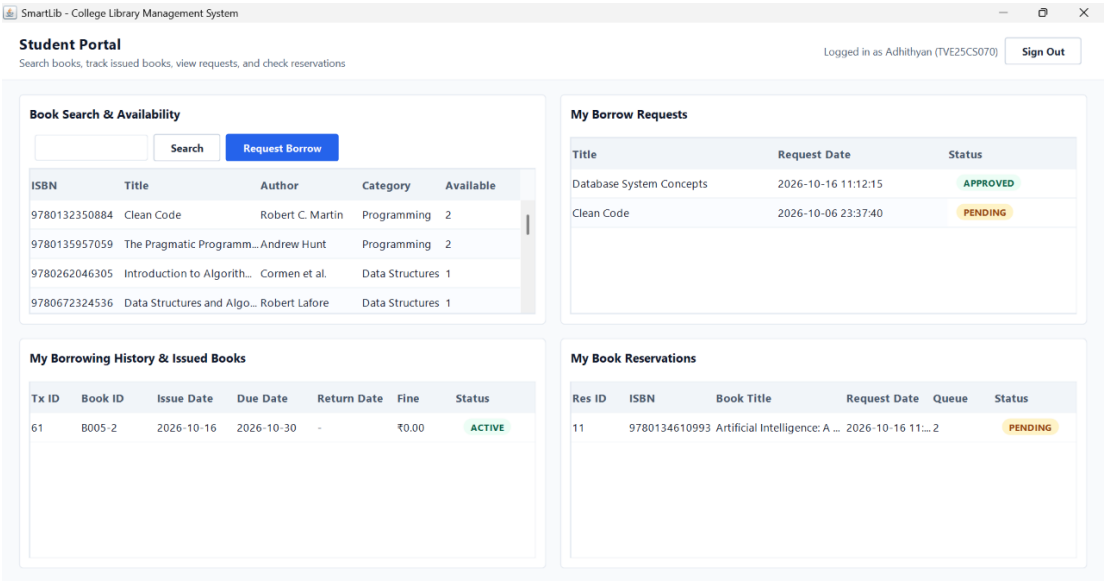



Figure 7.7: Student Self-Service Dashboard (`StudentDashboardPanel`)

## 7.3 Sample Operational Scenarios and Traces

To demonstrate system execution, three representative operational workflows are detailed below, tracing user input, service processing, and database state transitions.

### 7.3.1 Scenario A: Direct Book Issue at Circulation Desk

- **Initial Context:** Book `"B1001"` (*Effective Java*) is currently on the shelf with status `AVAILABLE`. Student `"TVE23CS001"` (*Alice Smith*) presents her college ID card at the desk.
- **Input Data:**
  - Student KTU ID: `"TVE23CS001"`
  - Book Accession ID: `"B1001"`
  - Librarian Session: ID 1 (`admin`)
- **Processing Logic:**
  1. `BorrowService.issueBook()` verifies that `Student` exists and status is `ACTIVE`.
  2. Verifies that `Book.getStatus() == BookStatus.AVAILABLE`.
  3. Generates issue date (`2026-10-23`) and due date (Today+14 days = 2026-11-06).
  4. Inserts a record into `borrow_transactions` with status `'ACTIVE'`.
  5. Executes `bookDAO.updateStatus("B1001", BookStatus.ISSUED)`.
- **Output / Result:** A success dialog displays *"Book issued successfully! Due date: 2026-11-06"*. The book's status pill in the catalog immediately changes from Green (`AVAILABLE`) to Amber (`ISSUED`), and the transaction appears in the Active Borrowings table.

### 7.3.2 Scenario B: Overdue Book Return with Fine Calculation and Reservation Alert

- **Initial Context:** Book `"B1003"` (*Domain-Driven Design*) was issued to `"TVE23CS001"` with due date `2023-10-15`. Another student `"TVE23CS002"` placed a reservation on that ISBN. The book is returned today.
- **Input Data:** Book Accession ID: `"B1003"`.
- **Processing Logic:**
  1. `BorrowService.returnBook("B1003", 1)` retrieves the active transaction record.
  2. Dynamic fine engine calculates overdue days: Overdue = Current Date − Due Date.

3. Accrued fine computed as Days Overdue × Rs. 10.00.
4. Transaction updated in database: status set to `'COMPLETED'`, return date set to current date, assessed fine recorded.
5. Book status reset to `AVAILABLE`.
6. `ReservationDAO.findOldestPendingByIsbn()` detects that a pending reservation exists for `"TVE23CS002"`.
7. The service appends an attention notice to the return confirmation message.

- **Output / Result:** The return is confirmed displaying the fine. An alert dialog informs the librarian: *"ATTENTION: This book is reserved!\nNext Student in Queue: TVE23CS002\nPlease set this book aside."*

### 7.3.3 Scenario C: Student Deletion Safeguard and Audit Protection

- **Initial Context:** A librarian attempts to delete student record `"TVE23CS001"` (*Alice Smith*) from the Student Directory desk. The student currently has an active borrowed book and historical loan records.
- **Input Data:** Select row `"TVE23CS001"` in `StudentPanel`, click *"Delete Student"*.
- **Processing Logic:**
  1. `StudentService.deleteStudent()` queries the student's associated records across `BorrowTransactionDAO`, `BorrowRequestDAO`, and `ReservationDAO`.
  2. Identifies 1 active borrowed book and associated historical circulation records.
  3. Builds an itemized list of blocking reasons to prevent foreign key errors and preserve historical records.
  4. Throws a descriptive `LibraryException` instructing the user to deactivate rather than delete.
- **Output / Result:** An alert dialog appears: *"Cannot permanently delete student TVE23CS001: Related records exist (1 active borrowed book(s)...). To preserve borrowing history, please deactivate the student instead."* The student record remains intact in the database.

# Chapter 8

# Conclusion and Future Enhancements

## 8.1 Summary of Achievements

The development of **SmartLib** has produced a reliable, multi-tier College Library Management System that addresses core operational needs of academic circulation desks while applying object-oriented software engineering principles.

All primary project objectives have been fulfilled:

1. **Architectural Integrity:** The multi-tier architecture cleanly isolates user interface presentation (`gui`), institutional business logic (`service`), data access abstraction (`dao`), and relational persistence (MySQL), adhering to the Single Responsibility Principle and Dependency Inversion Principle [8, 10].

2. **Circulation Excellence:** The Circulation Desk has been augmented with real-time Active Borrowings Management, providing librarians with live visibility over all active loans, dynamic clock-aware overdue calculations (Rs. 10.00/day), multi-column search, and direct one-click returns.

3. **Fair Queueing and Reservation Alerts:** Title-level reservations mitigate double-booking conflicts via an automated FIFO pipeline that alerts staff upon book return.

4. **Cryptographic Security and Role Separation:** Passwords are protected using 12-round BCrypt salt hashing [6], SQL injection risks are mitigated through ubiquitous use of parameterized `PreparedStatement` queries [4], and database credentials are decoupled into external configuration files.

5. **Verification and Quality Assurance:** The core business workflows and graphical user interfaces were validated through manual functional testing, confirming

expected behavior across 38 manual UI test scenarios covering user authentication, input validations, circulation desks, active loan tracking, and inventory controls.

## 8.2 Educational Outcomes and Retrospective

Completing this course project under **PBCST304 (Object Oriented Programming)** provided profound practical insights into modern software engineering:

- **Practical Mastery of OOP Pillars:** The abstract modeling of `Person`, dynamic polymorphism across DAO implementations, encapsulation of entity invariants, and clean interface segregation transformed theoretical textbook concepts into practical design solutions.
- **Understanding Real-World State Management:** Managing Swing component lifecycles, understanding the Event Dispatch Thread (EDT), and handling UI table deselection behavior demonstrated the intricacies of event-driven desktop software.
- **Defensive Persistence Engineering:** Hands-on management of JDBC transaction lifecycles, connection pooling considerations, and normalization to 3NF highlighted the necessity of data integrity in multi-user applications.

## 8.3 Future Enhancements

While SmartLib provides a comprehensive desktop solution, several valuable enhancements can expand its capabilities in future iterations:

1. **Barcode and RFID Scanner Integration:** Physical accession tracking can be streamlined by integrating hardware USB or Bluetooth barcode/RFID scanners. Scanning a physical book barcode would instantly populate the accession ID in the issue and return desk, cutting transaction times to under two seconds.
2. **Automated Background Overdue Reminders (Email and SMS):** A recurring daemon or scheduled service could query active borrowings daily at midnight and dispatch automated email/SMS reminder alerts to students 48 hours prior to the due date, as well as immediate notifications upon entering overdue status.
3. **Inter-Library Campus Networking:** Expanding the single-campus relational schema into a distributed multi-branch architecture would permit inter-library loans between sister engineering institutions across Kerala under the KTU network.

4. **Web and Mobile Companion Portal:** Developing a lightweight RESTful API layer (using Spring Boot or Jakarta EE) would enable responsive mobile applications (Flutter or React Native), allowing students to search the catalog, view dues, and reserve books from their personal smartphones without visiting library terminals.

5. **Digital Fine Payment Gateway Integration:** Integrating digital payment gateways (UPI, QR-Code dynamic generation, or net banking) would enable students to settle accumulated late fines directly within the application, automatically updating transaction statuses to `COMPLETED` and clearing academic holds.

# Bibliography


[1] APJ Abdul Kalam Technological University, "Curriculum for B.Tech Degree in Computer Science and Engineering: Course PBCST304 Object Oriented Programming," Thiruvananthapuram, Kerala, India, 2023.

[2] Oracle Corporation, "Java Platform, Standard Edition 17 API Specification," https://docs.oracle.com/en/java/javase/17/docs/api/, 2021.

[3] ——, "Creating a GUI With JFC/Swing," https://docs.oracle.com/javase/tutorial/uiswing/, 2021.

[4] ——, "Java Database Connectivity (JDBC) API Documentation," https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/package-summary.html, 2021.

[5] ——, "MySQL 8.0 Reference Manual," https://dev.mysql.com/doc/refman/8.0/en/, 2023.

[6] D. Miller, "jBCrypt: An OpenBSD-style Blowfish Password Hashing Library in Java," https://www.mindrot.org/projects/jBCrypt/, 2020.

[7] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*, 1999, pp. 81–91.

[8] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley Professional, 1994.

[9] Oracle Corporation, "MySQL Connector/J 8.0 Developer Guide," https://dev.mysql.com/doc/connector-j/8.0/en/, 2023.

[10] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley Professional, 2018.

# Appendix A

# User Manual and Installation Instructions

This appendix provides a step-by-step technical guide for configuring, building, and running SmartLib on a fresh development machine.

## A.1 Prerequisites Checklist

- **Java SE Development Kit (JDK 17 or higher):** Ensure `JAVA_HOME` environment variable points to your JDK directory and `java -version` reports version 17 or above.
- **MySQL Community Server (Version 8.0 or higher):** Installed and running on default TCP port `3306`.
- **Git:** For cloning the repository.
- **Maven:** Optional. The repository includes an executable Maven Wrapper (`mvnw` for Linux/macOS, `mvnw.cmd` for Windows).

## A.2 Database Initialization

1. Open MySQL Command Line Client or MySQL Workbench:

```
CREATE DATABASE smartlib;
USE smartlib;

```

2. Execute the database schema definition located at `src/main/resources/sql/schema.sql` to generate all 7 relational tables.

3. Execute the sample data population script located at `src/main/resources/sample_data.sql` to seed default categories, books, student records, and librarian administrative accounts.

## A.3 Database Configuration

1. Navigate to `src/main/resources/`.
2. Create a copy of `database.properties.example` and save it as `database.properties`.
3. Update the database credentials to match your local MySQL configuration:

```
db.url=jdbc:mysql://localhost:3306/smartlib
db.user=root
db.password=YOUR_MYSQL_PASSWORD

```

## A.4 Building and Executing the Application

Open a terminal in the root repository folder:

1. **Compile the Application:**

```
# On Linux / macOS:
./mvnw compile

# On Windows:
.\mvnw.cmd compile

```

   Verify that the build completes with zero errors.

2. **Launch the GUI Desktop Application:**

```
# On Linux / macOS:
./mvnw exec:java -Dexec.mainClass="com.college.library.gui.MainApplication"

# On Windows:
.\mvnw.cmd exec:java "-Dexec.mainClass=com.college.library.gui.MainApplication"

```

## A.5 Default User Credentials

Table A.1: Default Seed Credentials for Verification

| Portal / Role | Identifier / Username | Default Password | Access Level |
|---|---|---|---|
| Librarian Portal | admin | password123 | Full Administrative Access |
| Student Portal | TVE23CS001 | password123 | Student Self-Service |
| Student Portal | TVE23CS002 | password123 | Student Self-Service |

## A.6 Troubleshooting Matrix

Table A.2: Common Setup Errors and Solutions

| Error Message | Root Cause | Corrective Action |
|---|---|---|
| Communications link failure | MySQL service is stopped or port is not 3306 | Verify that MySQL service is running (systemctl status mysql or services.msc). |
| Access denied for user 'root' | Incorrect password in configuration | Verify and update root password in src/main/resources/database.properties. |
| Unknown database 'smartlib' | Database not initialized | Run CREATE DATABASE smartlib; in MySQL Workbench. |
| Table 'smartlib.books' doesn't exist | Schema script not executed | Import and execute src/main/resources/sql/schema.sql. |

# Appendix B

# Selected Source Code Listings

## B.1 Generic Base Data Access Object: `CrudDAO.java`

```java
package com.college.library.dao;

import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface CrudDAO<T, ID> {
    void save(T entity) throws SQLException;
    Optional<T> findById(ID id) throws SQLException;
    List<T> findAll() throws SQLException;
    void update(T entity) throws SQLException;
    void delete(ID id) throws SQLException;
}
```

Listing B.1: Generic CrudDAO Interface

## B.2 Circulation Desk and Fine Engine: `BorrowService.java`

```java
package com.college.library.service;

import com.college.library.dao.*;
import com.college.library.dto.ActiveBorrowingDTO;
import com.college.library.exception.LibraryException;
import com.college.library.model.*;

import java.math.BigDecimal;
import java.sql.SQLException;
import java.time.Clock;
import java.time.LocalDate;
```

```
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class BorrowService {
    private final BorrowTransactionDAO transactionDAO;
    private final BookDAO bookDAO;
    private final StudentDAO studentDAO;
    private final ReservationDAO reservationDAO;
    private final BorrowRequestDAO borrowRequestDAO;
    private final Clock clock;

    public static final BigDecimal DAILY_FINE_RATE = BigDecimal.valueOf(10.00);

    public BorrowService(BorrowTransactionDAO txDAO, BookDAO bookDAO,
                         StudentDAO studentDAO, ReservationDAO resDAO,
                         BorrowRequestDAO reqDAO) {
        this(txDAO, bookDAO, studentDAO, resDAO, reqDAO, Clock.systemDefaultZone());
    }

    public BorrowService(BorrowTransactionDAO txDAO, BookDAO bookDAO,
                         StudentDAO studentDAO, ReservationDAO resDAO,
                         BorrowRequestDAO reqDAO, Clock clock) {
        this.transactionDAO = txDAO;
        this.bookDAO = bookDAO;
        this.studentDAO = studentDAO;
        this.reservationDAO = resDAO;
        this.borrowRequestDAO = reqDAO;
        this.clock = clock;
    }

    public void issueBook(String studentId, String bookId, int librarianId)
            throws LibraryException {
        try {
            Optional<Student> student = studentDAO.findById(studentId);
            if (student.isEmpty()) throw new LibraryException("Student not found.");
            if (!"ACTIVE".equalsIgnoreCase(student.get().getStatus())) {
                throw new LibraryException("Student account is not active.");
            }
```

```
52
53              Optional<Book> book = bookDAO.findById(bookId);
54              if (book.isEmpty()) throw new LibraryException("Book not
    found.");
55              if (book.get().getStatus() != BookStatus.AVAILABLE) {
56                  throw new LibraryException("Book is currently
    unavailable (" + book.get().getStatus() + ").");
57              }
58
59              LocalDate issueDate = LocalDate.now(clock);
60              LocalDate dueDate = issueDate.plusDays(14);
61
62              BorrowTransaction tx = new BorrowTransaction();
63              tx.setStudentId(studentId);
64              tx.setBookId(bookId);
65              tx.setLibrarianId(librarianId);
66              tx.setIssueDate(issueDate);
67              tx.setDueDate(dueDate);
68              tx.setFine(BigDecimal.ZERO);
69              tx.setStatus("ACTIVE");
70
71              transactionDAO.save(tx);
72              bookDAO.updateStatus(bookId, BookStatus.ISSUED);
73          } catch (SQLException e) {
74              throw new LibraryException("Database error during book
    issue: " + e.getMessage(), e);
75          }
76      }
77
78      public boolean returnBook(String bookId, int librarianId) throws
    LibraryException {
79          try {
80              Optional<BorrowTransaction> activeTx = transactionDAO.
    findActiveByBookId(bookId);
81              if (activeTx.isEmpty()) {
82                  throw new LibraryException("No active borrow
    transaction found for book ID: " + bookId);
83              }
84
85              BorrowTransaction tx = activeTx.get();
86              LocalDate returnDate = LocalDate.now(clock);
87              BigDecimal fine = calculateOverdueFine(tx.getDueDate());
88
89              tx.setReturnDate(returnDate);
90              tx.setFine(fine);
91              tx.setStatus("COMPLETED");
92
```

```
            transactionDAO.update(tx);
            bookDAO.updateStatus(bookId, BookStatus.AVAILABLE);

            // Check reservation queue
            Optional<Book> bookOpt = bookDAO.findById(bookId);
            if (bookOpt.isPresent()) {
                Optional<Reservation> pendingRes = reservationDAO.
    findEarliestPendingByIsbn(bookOpt.get().getIsbn());
                if (pendingRes.isPresent()) {
                    Reservation r = pendingRes.get();
                    r.setStatus("FULFILLED");
                    reservationDAO.update(r);
                    return true; // Alert GUI that reservation is
    fulfilled
                }
            }
            return false;
        } catch (SQLException e) {
            throw new LibraryException("Database error during return:
    " + e.getMessage(), e);
        }
    }

    public BigDecimal calculateOverdueFine(LocalDate dueDate) {
        LocalDate today = LocalDate.now(clock);
        if (today.isAfter(dueDate)) {
            long daysOverdue = ChronoUnit.DAYS.between(dueDate, today)
    ;
            return BigDecimal.valueOf(daysOverdue).multiply(
    DAILY_FINE_RATE);
        }
        return BigDecimal.ZERO;
    }
}
```

Listing B.2: Core Methods in BorrowService.java

## B.3 Centralized Design System: `UITheme.java`

```
package com.college.library.gui;

import javax.swing.*;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableCellRenderer;
import javax.swing.table.JTableHeader;
import java.awt.*;
```

```
8
9  public class UITheme {
10     public static final Color BG_APP = new Color(248, 250, 252);
11     public static final Color BG_CARD = Color.WHITE;
12     public static final Color PRIMARY = new Color(37, 99, 235);
13     public static final Color PRIMARY_HOVER = new Color(29, 78, 216);
14     public static final Color TEXT_DARK = new Color(15, 23, 42);
15     public static final Color TEXT_MUTED = new Color(100, 116, 139);
16     public static final Color BORDER = new Color(226, 232, 240);
17
18     public static final Font FONT_HEADER = new Font("Segoe UI", Font.
    BOLD, 22);
19     public static final Font FONT_TITLE = new Font("Segoe UI", Font.
    BOLD, 15);
20     public static final Font FONT_BODY = new Font("Segoe UI", Font.
    PLAIN, 13);
21     public static final Font FONT_BOLD = new Font("Segoe UI", Font.
    BOLD, 13);
22
23     public static JButton createPrimaryButton(String text) {
24         JButton btn = new JButton(text);
25         btn.setFont(FONT_BOLD);
26         btn.setForeground(Color.WHITE);
27         btn.setBackground(PRIMARY);
28         btn.setFocusPainted(false);
29         btn.setBorder(new EmptyBorder(8, 16, 8, 16));
30         btn.setCursor(new Cursor(Cursor.HAND_CURSOR));
31         return btn;
32     }
33
34     public static void styleTable(JTable table) {
35         table.setFont(FONT_BODY);
36         table.setRowHeight(36);
37         table.setShowGrid(false);
38         table.setIntercellSpacing(new Dimension(0, 0));
39         table.setSelectionBackground(new Color(239, 246, 255));
40         table.setSelectionForeground(TEXT_DARK);
41
42         JTableHeader header = table.getTableHeader();
43         header.setFont(FONT_BOLD);
44         header.setBackground(new Color(241, 245, 249));
45         header.setForeground(TEXT_DARK);
46         header.setBorder(BorderFactory.createMatteBorder(0, 0, 1, 0,
    BORDER));
47     }
48 }
```

Listing B.3: Centralized UI Theme Styling Engine